AI Hooks Explained

FRONTMATTER

```
title: AI Hooks Explained
video no: 19 of 30
series: Building an AI Business From Scratch
voice_reference: ../../Input/4. Resources/2. scripting_resources/voice_reference/nate_herk_voice_reference.md
video_type: concept
hook_type: Demonstration
script_versions: A + B
status: DRAFT
```

REFERENCE VIDEOS

- Automate workflows with hooks -- Claude Code Docs (https://code.claude.com/docs/en/hooks-guide) -- Anthropic -- The authoritative reference: use to verify hook types, trigger points, and configuration syntax
- Claude Code Hooks: Complete Guide to All 12 Lifecycle Events (https://claudefa.st/blog/tools/hooks/hooks-guide) -- ClaudeFast -- Watch for the complete event list and the "Stop hook as self-evaluation" pattern -- the most powerful use of hooks
- Hook-driven dev workflows with Claude Code (https://nick-tune.me/blog/2026-02-28-hook-driven-dev-workflows-with-claude-code/) -- Nick Tune -- Read for real production examples of hooks enforcing workflow steps that Claude would otherwise skip probabilistically

THE ONE THING

Hooks turn "I hope Claude does this" into "Claude always does this" -- they are how you make a workflow deterministic instead of probabilistic.

HOOK

Claude will usually run your tests. Claude will mostly format the output correctly. Hooks replace usually and mostly with always. Here is how.

TARGET VIEWER

Someone building Claude Code workflows who finds Claude occasionally skipping steps, forgetting to format correctly, or not running tests -- and wants a reliable fix that does not involve more prompting.

PAYOFF

After watching, the viewer knows what hooks are, how they work in Claude Code, and can implement their first hook to enforce one rule that currently depends on Claude "remembering" to do it.

BEAT STRUCTURE

Beat -- The Probabilistic Problem

Point: Language models are probabilistic -- they usually do what you ask but not always. For production workflows, "usually" is not good enough.

Visual: Same Claude Code session running twice with the same instruction: first time runs tests, second time skips them -- highlight the inconsistency

Target words: ~110w

Beat -- What Hooks Are

Point: Hooks are shell commands or scripts that fire automatically at specific lifecycle events in a Claude Code session: before a tool runs, after it runs, when Claude stops, and others.

Visual: Diagram of Claude Code session lifecycle -- hooks firing at labelled points: PreToolUse, PostToolUse, Stop, SubagentStop, Notification

Target words: ~170w

Beat -- From Probabilistic to Deterministic

Point: If Claude might forget to run tests, a PostToolUse hook runs them automatically after every file edit. The hook does not forget, does not get distracted, does not skip.

Visual: Terminal demo: hook fires automatically after a file edit -- runs the test suite, reports back to Claude -- Claude sees the results and continues

Target words: ~190w

Beat -- Hook Types and Use Cases

Point: PreToolUse: validate or block before Claude acts. PostToolUse: verify or process after Claude acts. Stop: evaluate when Claude finishes. SubagentStop: collect sub-agent outputs.

Visual: Four-row table: hook type | when it fires | common use case -- one line each, clean and scannable

Target words: ~160w

Beat -- Writing Your First Hook

Point: Identify one step Claude does inconsistently. Write a 3-line shell command that enforces it. Add it to .claude/settings.json as a Stop hook. Test on one session.

Visual: .claude/settings.json open: adding a Stop hook that runs a linter -- then demo of it firing and blocking Claude from finishing until the code passes

Target words: ~140w

Beat -- CTA Bridge

Point: Rules, skills, and hooks are your control layer. Next: Ollama -- how to run AI models completely locally on your machine, no cloud, no API key, no cost per query.

Visual: Preview clip from Ollama Setup

Target words: ~60w

ANALOGY BANK

- Hooks vs prompting: Like the difference between asking someone to check their work before submitting vs building a mandatory review step into the submission process. One depends on memory; the other does not.

- Lifecycle hooks: Like git hooks -- pre-commit runs before you commit, post-commit runs after. Claude Code hooks follow the same pattern but for AI agent events. If you have used git hooks, you already understand the concept.

THESIS LINE LOCATION

Beat 3 -- The insight is: hooks shift control from probabilistic (Claude decides whether to do it) to deterministic (the system ensures it happens) -- for anything that matters in production, that shift is not optional.

FRAMEWORK PHRASE

Always, not usually

SPECIFICITY BANK

- Hooks configured in: .claude/settings.json under the "hooks" key
- Available lifecycle events: PreToolUse, PostToolUse, Stop, SubagentStop, Notification
- Hooks can run: shell commands, Python scripts, other CLI tools
- A Stop hook returning non-zero exit code blocks Claude from finishing -- forces continued work
- Up to 15 distinct lifecycle events available (as of 2026)
- Official docs: code.claude.com/docs/en/hooks-guide

CONSTRAINTS

- Do not go into advanced hook patterns like LLM-as-evaluator -- keep it to simple shell command hooks
- Do not explain the full settings.json structure -- focus only on the hooks section
- Do not confuse with web hooks, React hooks, or other programming hooks -- make the distinction explicit early
- Do not go into hook security considerations in depth -- mention that hooks run as shell commands and leave it at that

CTA DIRECTION

Rules, skills, and hooks -- you have the full control layer. Next: Ollama -- how to run AI models completely locally on your machine with no cloud, no API key, and no cost per query.

PERSONAL ANGLE [DANIEL TO FILL]

Suggested: The first hook you added and what workflow inconsistency it fixed -- how often that step would have been skipped without it.

VULNERABILITY MOMENT [DANIEL TO FILL]

Suggested: A production workflow where Claude skipped a critical step -- what it cost, and the specific hook you added to make sure it never happens again.
Note: belongs in Beat 1